

# RELATÓRIO DE VALIDAÇÃO TÉCNICA

**Plataforma CST Public Insta — Gestão Multi-Cliente de Publicações Instagram**

*Frontend, Backend, UX/UI e Prontidão Operacional*

Julho de 2026

# 1. Sumário Executivo

Este relatório documenta a validação técnica completa do projeto “CST Public Insta”, uma plataforma multi-cliente para criação, aprovação e publicação de conteúdo no Instagram, com integrações a Google Drive, Meta Graph API e geração de legendas via IA (Gemini). A análise cobriu o código-fonte de frontend (React + Vite + Tailwind), o backend (Node/Express monolítico em server.ts), o schema de dados (Supabase/Postgres) e a documentação de especificação fornecida no próprio repositório.

A plataforma demonstra uma base funcional ampla e um desenho de produto bem pensado: fluxo completo de criação → aprovação → publicação, controle de perfis de acesso (Super Admin, Admin, Admin do Cliente, Aprovador, Criador, Visualizador), validação de mídia (imagem/vídeo) antes da publicação, editor de vídeo embutido, painel de configurações por cliente e um modo “Simulador” que permite operar sem publicar de fato no Instagram. Esse nível de maturidade funcional é o ponto forte do projeto.

Por outro lado, a validação identificou problemas de segurança e de arquitetura que, no estado atual, tornam o sistema não recomendado para uso operacional com dados e clientes reais sem correções prévias. O mais grave: o pacote de arquivos enviado para esta análise contém um arquivo .env.local com credenciais reais e ativas (chave de serviço do Supabase, segredo do app Meta, client secret do Google) — ou seja, segredos de produção estão fisicamente presentes no pacote do projeto. Além disso, o backend permite, em determinadas condições, autenticação por cabeçalhos HTTP não verificados e não valida que um usuário administrador de um cliente pertença de fato àquele cliente antes de liberar leitura/edição — uma falha de isolamento multi-tenant (IDOR).

*Recomendação imediata, independente do restante deste relatório: revogar e rotacionar agora as credenciais listadas na Seção 3 (Supabase, Meta/Facebook, Google), pois estiveram expostas neste pacote de arquivos.*

As seções seguintes detalham, com evidências extraídas diretamente do código, os achados por área (segurança/backend, UX/UI, aderência ao escopo funcional) e apresentam um plano de ação priorizado.

## 1.1 Classificação geral

| Dimensão                                                       | Situação atual                                                                                                       | Nota (0–5) |
|----------------------------------------------------------------|----------------------------------------------------------------------------------------------------------------------|------------|
| Cobertura funcional (o que o produto entrega)                  | Ampla — cobre todo o ciclo de vida do post e a operação multi-cliente                                                | 4,0        |
| Segurança e controle de acesso                                 | Falhas críticas de autenticação/autorização e vazamento de segredos                                                  | 1,5        |
| Arquitetura e qualidade de código (backend)                    | Monolito único de ~6.800 linhas, forte acoplamento, baixa testabilidade                                              | 2,5        |
| UX / usabilidade operacional                                   | Fluxos claros, mas com uso de alert()/confirm() nativos, pouca padronização de feedback e ausência de acessibilidade | 3,0        |
| UI / consistência visual                                       | Boa identidade visual e responsividade; densidade de texto muito pequena (10–11px) em vários pontos                  | 3,5        |
| Prontidão para produção real (publicação de fato no Instagram) | Depende de configuração; hoje opera majoritariamente em modo Simulador                                               | 2,5        |

# 2. Escopo e Método da Análise

A análise cobriu 100% dos arquivos de código-fonte do projeto (não apenas uma amostra), incluindo:

- Frontend: src/App.tsx e os 13 componentes em src/components/ (CreatePost, ApproveList, Dashboard, AdminDashboard, ClientsManagement, ClientUsersManagement, UsersManagement, SettingsSync, SimulatorLogs, HistoryList, LoginScreen, LoginGate, VideoEditor, VideoValidationResult) e as bibliotecas em src/lib/.
- Backend: server.ts (arquivo único com aproximadamente 6.800 linhas, concentrando toda a API REST, integrações externas, criptografia de segredos e regras de negócio).
- Dados: supabase/schema.sql (schema Postgres) e src/types.ts (contratos de dados compartilhados).
- Documentação de especificação incluída no próprio repositório (pasta /documentos), usada como referência do comportamento pretendido para comparação com o comportamento implementado.
- Configuração de build/deploy (package.json, vite.config.ts, vercel.json) e arquivos de ambiente.

A avaliação seguiu quatro lentes complementares, como solicitado: (i) validação técnica de backend, (ii) validação técnica de frontend, (iii) User Experience (UX) — fluxos, fricção, tratamento de erro e (iv) User Interface (UI) — consistência visual, responsividade, acessibilidade — sempre com foco no uso operacional real (equipe de marketing/social media usando a ferramenta no dia a dia, com múltiplos clientes e permissões).

### 3. Alerta Crítico: Credenciais Expostas no Pacote do Projeto

O arquivo .env.local incluído no .zip enviado contém valores reais (não são placeholders) para os seguintes serviços:

| Serviço         | Variável                         | Observação                                                                                            |
|-----------------|----------------------------------|-------------------------------------------------------------------------------------------------------|
| Supabase        | SUPABASE_SERVICE_ROLE_KEY        | Chave mestra — dá acesso total de leitura/escrita ao banco, ignorando qualquer política de segurança. |
| Supabase        | SUPABASE_ANON_KEY / SUPABASE_URL | Menos crítico isoladamente, mas identifica o projeto Supabase associado.                              |
| Meta / Facebook | META_APP_SECRET                  | Permite operações em nome do app Meta (OAuth, Graph API) se combinado com o META_APP_ID.              |
| Google          | GOOGLE_CLIENT_SECRET             | Permite operações OAuth em nome da aplicação Google (Drive).                                          |

*Ação recomendada, a ser tomada antes de qualquer outra correção: (1) rotacionar/revogar imediatamente as quatro credenciais acima nos respectivos painéis (Supabase, Meta for Developers, Google Cloud Console); (2) confirmar que .env.local nunca foi commitado no histórico do Git (o .gitignore já bloqueia isso corretamente, mas o arquivo circulou fora do Git, dentro do .zip); (3) padronizar o uso de um cofre de segredos (Vercel Environment Variables, Supabase Vault, ou similar) para todo ambiente, e nunca reenviar .env.local em pacotes, e-mails ou compactados do projeto.*

### 4. Validação de Backend

O backend é um servidor Express único (server.ts), com mais de 80 rotas REST, que concentra autenticação, autorização, orquestração de integrações externas (Google Drive, Meta/Instagram Graph API, Gemini) e um armazenamento em memória (MemoryStore) usado como camada de cache/estado, sincronizado com Supabase quando configurado.

#### 4.1 Achados de segurança e controle de acesso

| Área                                | Problema identificado                                                                                                                                                                                                                                                                                                                                                                                          | Impacto operacional                                                                                                                                                                                                                                                                | Recomendação                                                                                                                                                                                                                             | Sev |
|-------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-----|
| <b>Autenticação</b>                 | A função <code>getActingUserFromRequest()</code> aceita, quando não há um token Supabase válido, os cabeçalhos HTTP <code>x-user-email</code> / <code>x-user-name</code> para identificar o usuário — sem qualquer verificação de senha, assinatura ou posse. Se nenhum cabeçalho for enviado, o sistema cai em um usuário padrão (o e-mail <code>cmourasiga@gmail.com</code> ou o primeiro ADMIN encontrado). | Qualquer chamada direta à API (fora da aplicação, por ex. via Postman/curl) pode se passar por qualquer usuário cadastrado apenas informando o e-mail dele em um cabeçalho, ou obter acesso de administrador por padrão sem enviar nada.                                           | Remover completamente o fallback por cabeçalhos e o usuário padrão implícito em ambiente de produção. Exigir token Supabase válido ( <code>Authorization: Bearer</code> ) em 100% das rotas, sem exceção, e retornar 401 quando ausente. |     |
| <b>Autorização multi-tenant</b>     | Rotas como <code>/api/clientes/:clienteld/usuarios</code> , <code>/integracoes</code> , <code>/configuracoes</code> verificam apenas o perfil do usuário (ex.: ADMIN_CLIENTE), mas nunca verificam se esse usuário está de fato vinculado ao <code>:clienteld</code> informado na URL.                                                                                                                         | Um Admin do Cliente A pode, trocando o <code>clienteld</code> na própria URL/requisição, ler ou alterar usuários, integrações e configurações do Cliente B — vazamento de dados entre clientes (IDOR), que é o pior cenário possível numa plataforma explicitamente multi-cliente. | Criar uma função <code>assertUserBelongsToClient(actingUser, clienteld)</code> e aplicá-la como guarda em toda rota sob <code>/api/clientes/:clienteld/*</code> , exceto para SUPER_ADMIN. Cobrir com testes automatizados de regressão. |     |
| <b>Segredos em repouso</b>          | Arquivo <code>.env.local</code> com credenciais reais de produção incluído no pacote entregue (ver Seção 3).                                                                                                                                                                                                                                                                                                   | Exposição de credenciais de banco de dados e integrações a qualquer pessoa com acesso ao arquivo/pacote.                                                                                                                                                                           | Rotacionar as credenciais e adotar um cofre de segredos gerenciado; nunca distribuir <code>.env*</code> fora de canais controlados.                                                                                                      |     |
| <b>Proteção de API</b>              | Não há middleware de rate limiting, nem helmet (cabeçalhos de segurança HTTP), nem CORS explícito configurado no Express.                                                                                                                                                                                                                                                                                      | Rotas sensíveis (login indireto, reset de senha, webhooks do Meta, callbacks OAuth) ficam expostas a força bruta e abuso sem limites de tentativa.                                                                                                                                 | Adicionar <code>express-rate-limit</code> nas rotas de autenticação/senha/OAuth e <code>helmet()</code> para cabeçalhos de segurança padrão; revisar necessidade de CORS explícito conforme domínio de deploy.                           |     |
| <b>Row Level Security (RLS)</b>     | O <code>schema.sql</code> não define nenhuma política de RLS nas tabelas do Supabase; toda a proteção de dados depende exclusivamente da lógica do Express.                                                                                                                                                                                                                                                    | Se a anon key (pública, embutida no frontend) for usada em qualquer chamada direta ao Supabase no futuro, ou em caso de bug no backend, não há uma segunda camada de defesa no banco.                                                                                              | Habilitar RLS em todas as tabelas com políticas por <code>cliente_id</code> , mesmo que o acesso hoje passe só pelo backend — defesa em profundidade.                                                                                    |     |
| <b>Mensagens de erro</b>            | <code>respondWithError()</code> repassa <code>error.message</code> diretamente ao cliente ( <code>maskError</code> apenas converte para string, não filtra conteúdo sensível).                                                                                                                                                                                                                                 | Mensagens internas (ex.: detalhes de falha do Supabase/Graph API) podem vaziar informações de implementação para o usuário final.                                                                                                                                                  | Padronizar mensagens genéricas para o usuário final e manter o detalhe técnico somente nos logs internos (a tabela de logs já existe e pode ser reaproveitada).                                                                          |     |
| <b>Segredo padrão de assinatura</b> | <code>MEDIA_URL_SIGNING_SECRET</code> tem fallback hardcoded ('local-media-secret') quando a variável de ambiente não está definida.                                                                                                                                                                                                                                                                           | Se essa variável não for configurada em produção, todas as URLs de mídia assinadas usam uma chave previsível e conhecida                                                                                                                                                           | Remover o fallback; falhar a inicialização do servidor se o segredo não estiver definido em produção.                                                                                                                                    |     |

| Área | Problema identificado | Impacto operacional                  | Recomendação | Sev |
|------|-----------------------|--------------------------------------|--------------|-----|
|      |                       | publicamente (está no código-fonte). |              |     |

## 4.2 Arquitetura e qualidade de código

| Área                        | Problema identificado                                                                                                                                                                                                 | Impacto operacional                                                                                                                                                                                             | Recomendação                                                                                                                                                                                                                         | Sev |
|-----------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-----|
| <b>Estrutura</b>            | Todo o backend está em um único arquivo server.ts (~6.800 linhas), misturando configuração, criptografia, integrações externas, regras de negócio e definição de rotas.                                               | Alto custo de manutenção, revisão de código e onboarding de novos desenvolvedores; risco elevado de regressões, já que qualquer alteração local pode ter efeitos não previstos em outra parte do mesmo arquivo. | Modularizar em camadas: routes/, controllers/, services/ (google-drive, instagram, gemini), repositories/ (acesso a dados) e um núcleo de auth/ separado. Pode ser feito de forma incremental, começando pelas integrações externas. |     |
| <b>Estado em memória</b>    | Existe um MemoryStore que guarda clientes, posts, usuários etc. em memória do processo Node, usado como cache e fallback quando o Supabase não está configurado.                                                      | Em ambientes serverless (o projeto já está preparado para Vercel via /api), instâncias diferentes de função não compartilham memória — dados podem parecer 'sumir' ou ficar inconsistentes entre requisições.   | Tratar o MemoryStore explicitamente como modo de desenvolvimento/demonstração local; documentar essa limitação e garantir que produção sempre exija Supabase configurado e íntegro.                                                  |     |
| <b>Testes automatizados</b> | Não foi encontrado nenhum diretório ou arquivo de testes (unitários, integração ou end-to-end) no projeto.                                                                                                            | Qualquer alteração de regra de negócio (ex.: cálculo de status de aprovação, validação de mídia) depende de teste manual, o que é arriscado numa API com mais de 80 rotas.                                      | Introduzir testes automatizados a partir das rotas mais críticas: autenticação, isolamento multi-tenant e o fluxo aprovar/rejeitar/publicar.                                                                                         |     |
| <b>Encoding de arquivos</b> | Diversos trechos do server.ts exibem caracteres corrompidos (ex.: 'Ã©', 'Ã¶') nas mensagens em português, indicando problema de codificação de caracteres (UTF-8/Latin-1) salvo em algum ponto do pipeline de edição. | Mensagens de erro e log ilegíveis para operadores, prejudicando o diagnóstico durante incidentes.                                                                                                               | Normalizar todos os arquivos-fonte para UTF-8 sem BOM e configurar o editor/IDE e o pipeline de build para preservar a codificação.                                                                                                  |     |
| <b>Artefatos no pacote</b>  | O .zip entregue inclui a pasta node_modules/ (dezenas de milhares de arquivos, ~75 MB) e a pasta dist/ (build gerado), que não deveriam fazer parte do controle de versão nem da entrega.                             | Dificulta a revisão e a distribuição do projeto; aumenta o risco de distribuir binários/artefatos desatualizados junto ao código-fonte.                                                                         | Confirmar que node_modules e dist estão no .gitignore (já estão) e reforçar o processo de exportação/zip do projeto para não incluí-los.                                                                                             |     |

## 4.3 O que o backend efetivamente entrega hoje

Do ponto de vista funcional, a API cobre corretamente: CRUD de posts com rascunho/edição, fluxo de submissão → aprovação/rejeição → publicação, validação de mídia (imagem e vídeo) no servidor antes de liberar publicação, geração de legendas com IA (Gemini), integração OAuth com Google Drive e com Instagram/Facebook (Meta Graph API), sincronização de insights (métricas de posts publicados), gestão de clientes, usuários e permissões por cliente, e

um log de auditoria de alterações de parâmetros sensíveis (ParametroAuditoria) — um diferencial positivo relevante para compliance.

O modo de operação é controlado por cliente (modo\_operacao: SIMULADOR ou REAL) e só passa a REAL quando o Supabase está corretamente configurado, o schema está íntegro e a URL pública da aplicação é acessível. Esse desenho é positivo (evita publicar de fato por engano), mas também significa que, sem essa configuração completa por cliente, o valor entregue hoje em produção real ainda é o de uma ferramenta de organização e aprovação de conteúdo — a publicação efetiva no Instagram depende de uma etapa de configuração que precisa ser validada cliente a cliente.

## 5. Validação de Frontend

### 5.1 Stack e organização

O frontend usa React 19 + Vite 6 + Tailwind 4, com ícones lucide-react e gráficos via recharts. A estrutura é simples e direta: um App.tsx central controla navegação por estado (sem React Router) e renderiza 13 componentes de tela conforme o perfil do usuário logado. Para o tamanho atual do produto isso funciona, mas a ausência de um roteador de verdade tem custos práticos listados abaixo.

| Área                   | Problema identificado                                                                                                                                                                                           | Impacto operacional                                                                                                                                                                                                                | Recomendação                                                                                                                                                                        | Sev |
|------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-----|
| Navegação (roteamento) | A navegação entre telas é feita trocando um estado currentScreen em memória (useState), sem uso de URL/React Router.                                                                                            | Não é possível compartilhar um link direto para uma tela específica (ex.: 'Moderação'), usar o botão Voltar do navegador, nem abrir duas telas em abas separadas. Ao atualizar a página (F5), o usuário sempre volta ao Dashboard. | Migrar para React Router (ou similar) mapeando cada tela para uma rota real (/aprovacao, /config, /usuarios etc.), preservando o controle de acesso por perfil como guarda de rota. |     |
| Gestão de estado/dados | Cada componente busca seus próprios dados via fetch direto (apiFetch) com useEffect, sem camada de cache/revalidação (ex.: React Query/SWR).                                                                    | Não há dedução de chamadas duplicadas, cache entre telas, nem tratamento padronizado de 'stale data'; o polling de 5s e 15s no App.tsx roda mesmo quando o usuário não está na tela que precisa desse dado.                        | Adotar uma biblioteca de data-fetching (TanStack Query) para cache, refetch inteligente e cancelamento de chamadas obsoletas.                                                       |     |
| Tipagem de status      | src/types.ts define PostStatus com pares redundantes (ex.: 'APROVADA' e 'APROVADO', 'PUBLICADA' e 'PUBLICADO', 'AGENDADA' e 'AGENDADO'), sugerindo duas convenções de nomenclatura convivendo no mesmo sistema. | Risco real de comparações de status falharem silenciosamente em algum componente (ex.: verificar só 'APROVADA' e nunca 'APROVADO'), gerando posts que 'somem' de uma lista por engano.                                             | Unificar em uma única convenção de nomenclatura (ex.: sempre no feminino, concordando com 'publicação') e migrar dados/lógica existentes com um script único.                       |     |
| Feedback ao usuário    | Erros e confirmações usam alert() e window.confirm() nativos do navegador (encontrados em CreatePost, ApproveList e UsersManagement).                                                                           | Interrompem o fluxo de forma abrupta, não seguem a identidade visual do produto, não funcionam bem em mobile/PWA e não ficam                                                                                                       | Substituir por um sistema de toast/notificação e modais de confirmação consistentes com o design system já usado no restante da aplicação.                                          |     |

| Área | Problema identificado | Impacto operacional                            | Recomendação | Sev |
|------|-----------------------|------------------------------------------------|--------------|-----|
|      |                       | registrados na tela para o usuário ler depois. |              |     |

### 5.2 Pontos fortes de frontend

- Upload de mídia com drag-and-drop, pré-visualização de imagem/vídeo e leitura de metadados (dimensão, duração, orientação) diretamente no navegador antes do envio.
- Editor de vídeo embutido (recorte, geração de thumbnail) usando FFmpeg no navegador (@ffmpeg/ffmpeg), evitando dependência de processamento server-side para essa etapa.
- Pannel de geração de legenda por IA integrado ao formulário de criação, com preview do post ao vivo simulando o feed do Instagram.
- Separação clara de telas por responsabilidade (Dashboard, Criar, Aprovação, Histórico, Configurações, Usuários, Logs), alinhada ao vocabulário do negócio (perfis em português: Criador, Aprovador, Administrador etc.).

## 6. Validação de UX (Experiência de Uso Operacional)

Do ponto de vista de quem usa a ferramenta no dia a dia — social media, aprovador, administrador de conta — os principais fluxos (criar rascunho, enviar para aprovação, aprovar/rejeitar, publicar, consultar histórico) existem e seguem uma lógica compreensível. Os pontos abaixo afetam a eficiência e a confiança do usuário operacional:

| Área                         | Problema identificado                                                                                                                                                                                                                              | Impacto operacional                                                                                                                                                              | Recomendação                                                                                                                                                           | Sev |
|------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-----|
| Confirmações críticas        | Ações destrutivas (excluir usuário, redefinir senha, excluir rascunho) usam <code>window.confirm()</code> , um diálogo do navegador sem contexto adicional (ex.: não mostra 'X posts serão perdidos').                                             | Aumenta o risco de exclusões acidentais ou, ao contrário, gera desconfiança por parecer um alerta de erro do navegador.                                                          | Modal de confirmação customizado, exibindo o nome do item afetado e, quando aplicável, as consequências (ex.: 'Este rascunho tem mídia anexada que será perdida').     |     |
| Estado vazio / sem permissão | Ao acessar uma tela sem permissão, o usuário vê um cartão genérico 'Acesso restrito' central — não há prevenção no próprio menu, que já esconde a opção quando não permitida (isso é positivo), mas rotas ainda são acessíveis via estado interno. | Consistente na maior parte, mas pequenas inconsistências de mensagem podem confundir o usuário sobre por que não vê determinada função.                                          | Manter o padrão atual (bom) e garantir mensagens padronizadas explicando qual perfil é necessário para a ação.                                                         |     |
| Recuperação de senha         | Não foi encontrado fluxo de 'Esqueci minha senha' na tela de login; a redefinição de senha só aparece como ação administrativa dentro de Gestão de Usuários.                                                                                       | Usuário operacional que esquece a senha depende de um administrador disponível para redefinir — gera fricção e possível bloqueio de trabalho em horários críticos de publicação. | Adicionar fluxo de autoatendimento de recuperação de senha via Supabase Auth (reset por e-mail).                                                                       |     |
| Feedback de progresso longo  | Upload de mídia, geração de legenda por IA e processamento de vídeo mostram spinners, mas sem indicação de progresso percentual nem tempo estimado, especialmente sensível em uploads de vídeo maiores.                                            | Em conexões mais lentas (comum em operação de campo/mobile), o usuário não sabe se o processo travou ou está avançando, podendo cancelar/repetir a ação sem necessidade.         | Adicionar barra de progresso real para uploads (o fetch/XHR permite) e mensagens intermediárias no processamento de vídeo (ex.: 'Recortando... Gerando thumbnail...'). |     |

| Área                      | Problema identificado                                                                                                                                                                      | Impacto operacional                                                                                                                                            | Recomendação                                                                                                                                             | Sev |
|---------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------|-----|
| <b>Polling automático</b> | O contador de pendências (badge) é atualizado a cada 5 segundos e a 'simulação de tick' a cada 15 segundos, continuamente, enquanto o app estiver aberto, independentemente da tela ativa. | Consumo desnecessário de rede/bateria em uso prolongado (ex.: aba aberta o dia todo), e maior carga no backend conforme a base de usuários simultâneos cresce. | Mover para atualização orientada a eventos (Supabase Realtime/webhooks) ou, no mínimo, pausar o polling quando a aba perde o foco (Page Visibility API). |     |

## 7. Validação de UI (Interface e Consistência Visual)

| Área                             | Problema identificado                                                                                                                                                                                                  | Impacto operacional                                                                                                                                                                                                                                                 | Recomendação                                                                                                                                                                                                                      | Sev |
|----------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-----|
| <b>Acessibilidade</b>            | Nenhum atributo aria-* ou role= foi encontrado em todo o código de frontend (busca exaustiva nos componentes); campos de formulário dependem apenas de <label> visual associado por proximidade.                       | Usuários que dependem de leitor de tela ou navegação por teclado terão dificuldade real de operar o sistema; também é um risco de conformidade (ex.: WCAG) caso a plataforma seja usada por clientes com exigências de acessibilidade.                              | Auditoria de acessibilidade dedicada: labels associados via htmlFor/id, aria-label em ícones/botões sem texto, foco visível e navegação 100% por teclado nos fluxos críticos (login, criar, aprovar).                             |     |
| <b>Densidade tipográfica</b>     | Uso extensivo de classes Tailwind como text-[9px], text-[10px], text-[11px] e text-xs em praticamente todos os componentes, inclusive em informações importantes (e-mail do usuário, status, valores de configuração). | Texto muito pequeno para uso prolongado em tela (fadiga visual), problemas de leitura em monitores de alta densidade e em dispositivos móveis, e conflito direto com boas práticas de acessibilidade (tamanho mínimo recomendado de 14px/12pt para corpo de texto). | Definir uma escala tipográfica mínima (ex.: 12px para textos auxiliares, 14px para corpo, 16px+ para dados primários) e aplicar de forma consistente via tokens do design system, eliminando tamanhos arbitrários abaixo de 11px. |     |
| <b>Responsividade</b>            | A aplicação trata bem breakpoints mobile/desktop (menu inferior fixo em mobile, sidebar fixa em desktop, grid adaptativo no formulário de criação).                                                                    | Ponto forte — reduz risco de retrabalho ao liberar uso em celular para aprovação rápida em campo.                                                                                                                                                                   | Manter o padrão atual; validar apenas os componentes mais densos (SettingsSync, tabelas de usuários) em telas pequenas, onde tabelas tendem a exigir rolagem horizontal.                                                          |     |
| <b>Identidade visual / marca</b> | Uso consistente de cores de marca (brand-primary/secondary/dark) e logo através de src/lib/branding.ts, com nome de produto configurável.                                                                              | Ponto forte — facilita white-label para diferentes clientes/contas.                                                                                                                                                                                                 | Nenhuma ação necessária; recomenda-se apenas documentar a paleta e tipografia como um pequeno guia de estilo para consistência em futuras telas.                                                                                  |     |

## 8. Aderência entre a Especificação e o que Foi Construído

O repositório traz uma documentação de especificação bastante detalhada (pasta /documentos, ~6.000 linhas somadas, cobrindo plataforma multi-cliente, telas, módulo de parametrizações, OAuth do Google Drive e integração com Instagram). Comparando essa especificação com o código implementado:

- Os principais pilares descritos (multi-cliente, perfis de acesso, parametrizações por cliente com opção de herdar do sistema, integração Google Drive e Instagram via OAuth) estão de fato implementados no backend, o que indica que a especificação foi usada como guia real de construção — não é documentação divorciada do código.



- A camada de segurança descrita implicitamente pela especificação (dados isolados por cliente) não é garantida pelo código atual, conforme detalhado na Seção 4.1 — o maior desvio entre 'o que foi projetado' e 'o que foi entregue'.
- Funcionalidades de auditoria de parâmetros (ParametroAuditoria) e testes de integração (endpoints /testar) foram implementadas e vão além do mínimo esperado — um ponto positivo de qualidade de engenharia que não aparece explicitamente nos documentos revisados.

## 9. O Que a Plataforma Entrega Hoje, na Prática

Considerando o código como está, hoje o sistema é capaz de, de ponta a ponta:

- Cadastrar e gerenciar múltiplos clientes, cada um com sua própria marca, integrações e parametrizações.
- Criar posts (imagem, vídeo ou reels) com upload, validação técnica de mídia e geração de legenda assistida por IA.
- Rodar um fluxo de aprovação com papéis distintos (quem cria não é obrigatoriamente quem aprova/publica).
- Registrar histórico e logs de auditoria de cada ação relevante, útil para rastreabilidade em caso de disputa ou erro de publicação.
- Operar em modo Simulador (sem publicar de fato) e, quando cada cliente tiver Google Drive e Instagram corretamente conectados via OAuth, publicar de fato e sincronizar métricas básicas de desempenho (visualizações, alcance, curtidas, comentários).

Ou seja: como produto de gestão e governança de conteúdo (organizar, validar e aprovar o que será postado), a entrega já é sólida hoje. Como produto de publicação automatizada real em múltiplos clientes simultâneos, a entrega depende (a) da configuração ponta a ponta das integrações por cliente e (b) da correção das falhas de segurança da Seção 4, sem as quais não é seguro operar com dados de clientes reais e credenciais reais de Instagram/Meta.

## 10. Plano de Ação Priorizado

### 10.1 Imediato (antes de qualquer uso com dados reais)

- Rotacionar todas as credenciais listadas na Seção 3 (Supabase, Meta, Google).
- Remover o fallback de autenticação por cabeçalho (x-user-email/x-user-name) e o usuário padrão implícito em produção.
- Implementar a verificação de vínculo usuário↔cliente (assertUserBelongsToClient) em todas as rotas /api/clientes/:clientId/\*.

### 10.2 Curto prazo (próximas semanas)

- Adicionar rate limiting e cabeçalhos de segurança HTTP (helmet) em toda a API.
- Unificar a nomenclatura de status de post (eliminar pares redundantes em PostStatus).
- Substituir alert()/confirm() por componentes de notificação e confirmação próprios do design system.
- Implementar fluxo de recuperação de senha por e-mail.

### 10.3 Médio prazo (próximo trimestre)

- Modularizar o backend (routes/services/repositories), com cobertura de testes automatizados começando pelos fluxos de autenticação, isolamento multi-cliente e aprovação/publicação.
- Migrar a navegação do frontend para um roteador real (URLs e histórico do navegador).
- Habilitar Row Level Security no Supabase como camada adicional de defesa.
- Revisão de acessibilidade (labels, aria-\*, navegação por teclado) e ajuste da escala tipográfica mínima.

## 10.4 Contínuo

- Adotar cache/revalidação de dados no frontend (ex.: TanStack Query) para reduzir polling e melhorar performance percebida.
- Padronizar codificação de arquivos (UTF-8) em todo o repositório.
- Manter a documentação de especificação atualizada conforme o código evolui, já que hoje ela é fiel ao produto — um ativo raro que vale a pena preservar.

## 11. Conclusão

O projeto “CST Public Insta” tem uma proposta de produto bem definida e uma implementação funcional que cobre corretamente o ciclo de vida de uma publicação multi-cliente, com diferenciais como auditoria de parâmetros, editor de vídeo embutido e modo simulador para operação segura antes de ir ao ar. Esses pontos devem ser preservados e valorizados na evolução do produto.

Ao mesmo tempo, para uso operacional com clientes e dados reais, o sistema hoje apresenta riscos de segurança que precisam ser tratados antes — em especial a autenticação por cabeçalho não verificado, a ausência de checagem de pertencimento de usuário ao cliente (isolamento multi-tenant) e a exposição de credenciais reais no pacote entregue. Corrigidos esses pontos críticos e endereçados os itens de curto/médio prazo listados na Seção 10, a plataforma tem uma base sólida para operar com confiança em produção.